Davies-Bouldin Index

Two Fully Worked Examples | Cluster Validity | Week 14

What this document covers

Why clustering needs a quality score | What variance means | The DB index formula explained

Full worked example: K=3 (three clusters) | Comparison: K=2 vs K=3 | How to find optimal K

Part 1 — Why Do We Need the Davies-Bouldin Index?

After you run K-Means or Hierarchical clustering, the algorithm simply gives you clusters. But it cannot answer the two most important questions:

Problem 1: What is the correct number of clusters, K?

The algorithm does whatever K you tell it. K=2? Fine. K=10? Also fine. But which is actually correct? The algorithm has no way to tell you.

Problem 2: Which clustering result is better?

K-Means gives different results depending on the random starting centroids. How do you compare two different runs and decide which one is better?

The solution is a **clustering quality index** — a single number that scores how good a clustering is. We compute this score for K=2, K=3, K=4, and so on. The K that gives the **lowest score** is the best choice.

The Davies-Bouldin (DB) index is one such measure. It was proposed by David Davies and Donald Bouldin in 1979. The key rule is simple:

GOLDEN RULE: Lower DB value = Better clustering

The DB index measures whether clusters are:

- **Tight inside** (low variance within each cluster — points are close together)
- **Well separated** from each other (large distance between cluster centres)

A clustering where clusters are tight AND far apart will have a low DB score. A clustering where clusters are messy AND overlapping will have a high DB score.

Part 2 — Key Concepts You Must Know First

2.1 Intra-Cluster Variance (Spread Inside a Cluster)

Variance measures how spread out the data points inside a single cluster are. If all points in a cluster are very close to the centre, variance is low. If they are scattered far from the centre, variance is high.

```
var(C)  =  (1 / N-1)  x  SUM of  (xi - x_mean)^2
```

Where: **N** = number of points in the cluster, **x_mean** = mean (average) of the cluster, **xi** = each individual data point

Important special case: If a cluster has only ONE data point, its variance is ZERO. The single point is its own centre — there is no spread. Also, the centroid of a single-point cluster is just that point itself.

2.2 Centroid (Centre of a Cluster)

The centroid is the mean (average) of all points in a cluster. For 1D data it is simply the arithmetic mean:

```
centroid  =  (x1 + x2 + x3 + ... + xN)  /  N
```

2.3 Inter-Cluster Distance (Gap Between Two Clusters)

The distance between two clusters is the absolute difference between their centroids. For 1D data:

```
distance(Ci, Cj)  =  | centroid_i  -  centroid_j |
```

This is just how far apart the two cluster centres are on the number line. A larger distance means the clusters are well separated — which is what we want.

Part 3 — The Davies-Bouldin Formula

The DB index is computed in three sub-steps. Each builds on the previous one.

Formula 1 — The Rij Score (Confusion Score Between Two Clusters)

```
Rij  =  [ var(Ci) + var(Cj) ]  /  || ci - cj ||
```

```
where   i   NOT EQUAL TO   j
```

This formula computes a **"confusion score"** between cluster i and cluster j. Let us break it down piece by piece:

Part of Formula	What It Measures	What it means for quality
var(Ci)	How spread out cluster i is internally	High = messy cluster = BAD
var(Cj)	How spread out cluster j is internally	High = messy cluster = BAD
var(Ci) + var(Cj)	Total internal messiness of BOTH clusters	Higher sum = worse
|| ci - cj ||	Distance between the two cluster centres	Larger = more separated = GOOD
Rij (the ratio)	Confusion between cluster i and cluster j	Higher Rij = harder to tell apart = BAD

Analogy: Think of Rij as a confusion score. If two students are both bad at studying (high variance) AND sit right next to each other (low distance), you will confuse which group they belong to. High confusion = high Rij = bad clustering.

Key property: Rij = Rji (symmetric). The confusion between cluster 1 and cluster 2 is the same as the confusion between cluster 2 and cluster 1.

Formula 2 — Ri (Worst Neighbour Score for Each Cluster)

```
Ri   =   max ( Rij )     for all j   NOT EQUAL TO   i
```

For each cluster i, we look at all its Rij scores with every other cluster j, and take the **maximum**. This captures the **worst case** — the neighbouring cluster that is hardest to distinguish from cluster i.

Why take the maximum? Because if cluster i is clearly separated from 4 other clusters but badly confused with just 1 cluster, that 1 bad case is what matters. The maximum picks up the most problematic pair.

Formula 3 — DB (Final Index — Average of Worst Cases)

```
DB  =  (1/K)  x  SUM of Ri     (sum from i=1 to K)
```

The final DB index is the **average of all Ri values** across all K clusters. It summarises the overall quality of the entire clustering in one number.

To find the optimal K: compute DB for K=2, K=3, K=4, ..., K=N-1. Plot the values. The K with the **minimum DB** is the best choice.

Part 4 — Full Worked Example (K = 3 Clusters)

Dataset: `x = { 3, 7, 10, 17, 18, 20 }`

Clustering: K = 3 clusters | Cluster 1 = {3} | Cluster 2 = {7, 10} | Cluster 3 = {17, 18, 20}

Goal: Compute the DB index for this 3-cluster solution.

STEP 1

Compute the Variance of Each Cluster

var(C) = (1/N-1) x SUM of (xi - x_mean)^2

Cluster 1 = { 3 } (only one data point)

When a cluster has only one data point, there is nothing to spread around. Variance = 0 by definition. The centroid is the point itself.

N	`1 point only`
Mean (centroid)	`x_mean = 3 / 1 = 3`
Variance	`var(C1) = 0 (only one point, no spread possible)`

var(C1) = 0 | centroid c1 = 3

Cluster 2 = { 7, 10 } (two data points)

With two points, we first find their mean, then measure how far each point is from that mean, square those distances, sum them up, and divide by N-1.

N	`2 points: x1 = 7, x2 = 10`
Step 1 — Mean	`x_mean = (7 + 10) / 2 = 17 / 2 = 8.5`
Step 2 — Deviations	`(7 - 8.5) = -1.5 | (10 - 8.5) = +1.5`
Step 3 — Squared	`(-1.5)^2 = 2.25 | (+1.5)^2 = 2.25`
Step 4 — Sum	`2.25 + 2.25 = 4.50`
Step 5 — Divide by N-1	`var(C2) = 4.50 / (2-1) = 4.50 / 1 = 4.50`

var(C2) = 4.50 | centroid c2 = 8.5

Cluster 3 = { 17, 18, 20 } (three data points)

Same process with three points. More steps but same logic.

N	`3 points: x1=17, x2=18, x3=20`
Step 1 — Mean	`x_mean = (17 + 18 + 20) / 3 = 55 / 3 = 18.33`
Step 2 — Deviations	`(17-18.33) = -1.33 | (18-18.33) = -0.33 | (20-18.33) = +1.67`
Step 3 — Squared	`(-1.33)^2 = 1.77 | (-0.33)^2 = 0.11 | (+1.67)^2 = 2.79`
Step 4 — Sum	`1.77 + 0.11 + 2.79 = 4.67`
Step 5 — Divide by N-1	`var(C3) = 4.67 / (3-1) = 4.67 / 2 = 2.33`

var(C3) = 2.33 | centroid c3 = 18.33

Summary of Step 1 Results

Cluster	Members	N	Mean (Centroid)	Variance
C1	{3}	1	c1 = 3.00	var(C1) = 0.00
C2	{7, 10}	2	c2 = 8.50	var(C2) = 4.50
C3	{17, 18, 20}	3	c3 = 18.33	var(C3) = 2.33

STEP
2

Compute Rij for Every Pair of Clusters

Rij = [var(Ci) + var(Cj)] / || ci - cj ||

With K=3 clusters, we have three possible pairs: (1,2), (1,3), and (2,3). We compute Rij for each pair. Remember: R12 = R21, R13 = R31, R23 = R32.

Pair (1, 2) — Clusters C1 and C2

Numerator	`var(C1) + var(C2) = 0.00 + 4.50 = 4.50`

Denominator	`|| c1 - c2 || = | 3.00 - 8.50 | = | -5.50 | = 5.50`
R12	`R12 = 4.50 / 5.50 = 0.818`

R12 = R21 = 0.818

Pair (1, 3) — Clusters C1 and C3

Numerator	`var(C1) + var(C3) = 0.00 + 2.33 = 2.33`
Denominator	`|| c1 - c3 || = | 3.00 - 18.33 | = | -15.33 | = 15.33`
R13	`R13 = 2.33 / 15.33 = 0.152`

R13 = R31 = 0.152

Pair (2, 3) — Clusters C2 and C3

Numerator	`var(C2) + var(C3) = 4.50 + 2.33 = 6.83`
Denominator	`|| c2 - c3 || = | 8.50 - 18.33 | = | -9.83 | = 9.83`
R23	`R23 = 6.83 / 9.83 = 0.695`

R23 = R32 = 0.695

Summary of All Rij Values

Pair	var(Ci)	var(Cj)	Numerator	|| ci - cj ||	Rij
C1 & C2	0.00	4.50	0.00 + 4.50 = 4.50	| 3.00 - 8.50 | = 5.50	0.818
C1 & C3	0.00	2.33	0.00 + 2.33 = 2.33	| 3.00 - 18.33 | = 15.33	0.152
C2 & C3	4.50	2.33	4.50 + 2.33 = 6.83	| 8.50 - 18.33 | = 9.83	0.695

STEP 3	**Compute Ri — Worst Neighbour Score for Each Cluster** *Ri = max(Rij) for all j NOT EQUAL TO i*

For each cluster, find the maximum Rij value among all pairs that include that cluster. This is the "worst case" — the hardest-to-distinguish neighbour.

R1 — for Cluster 1

Cluster 1 appears in pairs (1,2) and (1,3). We look at both Rij values and take the maximum.

Pairs involving C1	`R12 = 0.818      and      R13 = 0.152`
R1 = max of these	`R1 = max(0.818, 0.152) = 0.818`

R1 = 0.818

R2 — for Cluster 2

Cluster 2 appears in pairs (1,2) and (2,3).

Pairs involving C2	`R21 = 0.818      and      R23 = 0.695`
R2 = max of these	`R2 = max(0.818, 0.695) = 0.818`

R2 = 0.818

R3 — for Cluster 3

Cluster 3 appears in pairs (1,3) and (2,3).

Pairs involving C3	`R31 = 0.152      and      R32 = 0.695`
R3 = max of these	`R3 = max(0.152, 0.695) = 0.695`

R3 = 0.695

Summary of All Ri Values

Cluster	Rij values available	Maximum taken	Ri
C1	R12 = 0.818 | R13 = 0.152	max(0.818, 0.152)	R1 = 0.818
C2	R21 = 0.818 | R23 = 0.695	max(0.818, 0.695)	R2 = 0.818
C3	R31 = 0.152 | R32 = 0.695	max(0.152, 0.695)	R3 = 0.695

STEP
4

Compute the Final DB Index

DB = (1/K) x SUM of Ri

Now we average all the Ri values. With K=3:

K	`K = 3 clusters`
Sum of all Ri	`R1 + R2 + R3 = 0.818 + 0.818 + 0.695 = 2.331`
DB = (1/K) x Sum	`DB = (1/3) x 2.331 = 2.331 / 3 = 0.777`

DB (K=3) = 0.777

This is the quality score for the 3-cluster solution.

Part 5 — Second Example (K = 2 Clusters) for Comparison

Same dataset: `x = { 3, 7, 10, 17, 18, 20 }`

New clustering: K = 2 clusters | Cluster 1 = {3, 7, 10} | Cluster 2 = {17, 18, 20}

Goal: Compute DB for K=2, then compare with the K=3 result to see which is better.

STEP 1 — **Compute Variance and Centroid for Each Cluster**

Cluster 1 = { 3, 7, 10 }

N	`3 points: x1=3, x2=7, x3=10`
Step 1 — Mean	`x_mean = (3 + 7 + 10) / 3 = 20 / 3 = 6.67`
Step 2 — Deviations	`(3-6.67) = -3.67 | (7-6.67) = +0.33 | (10-6.67) = +3.33`
Step 3 — Squared	`(-3.67)^2 = 13.47 | (0.33)^2 = 0.11 | (3.33)^2 = 11.09`
Step 4 — Sum	`13.47 + 0.11 + 11.09 = 24.67`
Step 5 — Divide by N-1	`var(C1) = 24.67 / 2 = 12.33`

var(C1) = 12.33 | centroid c1 = 6.67

Cluster 2 = { 17, 18, 20 } (same as before)

This is identical to Cluster 3 from our K=3 example. We already computed it.

var(C2) = 2.33 | centroid c2 = 18.33

STEP 2 — **Compute R12 — the Only Pair (K=2 means only one pair)**

With K=2, there is only one possible pair: (C1, C2).

Numerator	`var(C1) + var(C2) = 12.33 + 2.33 = 14.66`
Denominator	`|| c1 - c2 || = | 6.67 - 18.33 | = | -11.66 | = 11.66`
R12	`R12 = 14.66 / 11.66 = 1.257`

R12 = R21 = 1.257

STEP 3 **Compute Ri — Worst Neighbour Score**

With only 2 clusters, each cluster has exactly one neighbour. So Ri = Rij directly.

R1	`R1 = max(R12) = 1.257 (only one option)`
R2	`R2 = max(R21) = 1.257 (only one option)`

STEP 4 **Compute the Final DB Index**

K	`K = 2 clusters`
Sum of all Ri	`R1 + R2 = 1.257 + 1.257 = 2.514`
DB = (1/K) x Sum	`DB = (1/2) x 2.514 = 2.514 / 2 = 1.257`

DB (K=2) = 1.257

This is the quality score for the 2-cluster solution.

Part 6 — Comparison and Final Decision

We now have DB scores for two different clusterings of the same dataset. Let us compare them side by side.

Clustering	Clusters	DB Score	Verdict
K = 3	C1={3} C2={7,10} C3={17,18,20}	DB = 0.777	BETTER (lower DB)
K = 2	C1={3,7,10} C2={17,18,20}	DB = 1.257	WORSE (higher DB)

CONCLUSION: K = 3 is the better clustering

DB(K=3) = 0.777 < DB(K=2) = 1.257

Lower DB means tighter clusters that are better separated from each other.

Why is K=3 better? — The intuition

Look at the K=2 clustering. Cluster 1 = {3, 7, 10} has a variance of 12.33, which is very high. The numbers 3, 7, and 10 are not naturally close to each other — they span a large range. This is why the K=2 DB score is bad.

In the K=3 clustering, by separating {3} out on its own, we get a cleaner picture: {3} alone, {7,10} close together, and {17,18,20} close together. All three clusters are tighter, so the overall DB is lower.

How to find optimal K in practice

In real problems, you would compute DB for every possible value of K from 2 to N-1 and then plot the results:

1. Compute DB for K=2, K=3, K=4, ..., K=N-1
2. Plot K on the x-axis and DB score on the y-axis
3. Find the K with the minimum (lowest) DB score
4. That K is your optimal number of clusters

Note: For a dataset with N=10 points, we compute DB for K=2, 3, 4, 5, 6, 7, 8, 9 (that is K=N-1 values). We do NOT compute K=10 because that gives each point its own cluster, which is meaningless.

Part 7 — Quick Reference for Exam and Teaching

All Formulas in One Place

```
STEP 1:   var(C) = (1/N-1) x SUM of (xi - x_mean)^2

STEP 2:   Rij = [ var(Ci) + var(Cj) ] / || ci - cj ||
       (compute for EVERY pair i,j where i NOT EQUAL j)

STEP 3:   Ri = max ( Rij )   for all j not equal to i
              (take the MAXIMUM for each cluster)

STEP 4:   DB = (1/K) x SUM of Ri   (from i=1 to K)
                  (average of all Ri values)

RULE:     Lower DB = Better clustering
```

Common Mistakes to Avoid

Mistake	Correct approach
Forgetting that Rij is symmetric (Rij = Rji)	You do NOT need to compute R12 and R21 separately — they are the same value
Using Ri instead of the maximum Rij	Ri is NOT just any Rij — it is specifically the MAXIMUM Rij for that cluster
Computing variance of a single-point cluster as non-zero	If N=1, variance = 0 and centroid = the single point itself
Using N instead of N-1 in variance formula	Always divide by N-1 (sample variance), not N (population variance)
Thinking higher DB is better	NO — LOWER DB is ALWAYS better. This is the most common confusion.

The 5 Rules to Remember

5. **Lower DB = better clustering** (never the other way around)
6. **DB can never be negative** (variance >= 0, distance > 0, so ratio >= 0)
7. **Tighter clusters → lower DB** (reducing variance reduces the Rij numerator)
8. **More separated clusters → lower DB** (increasing centroid distance increases the Rij denominator)
9. **Compute DB for K=2 to K=N-1** (then pick the K with the minimum DB)